

Compiler Design - Module 1

Intermediate Level Study Guide

Input Buffering • Transition Diagrams • Intermediate Code Generation • Code Optimization

1■■ Input Buffering (Intermediate Level)

■ Why Input Buffering is needed

During lexical analysis, the compiler reads characters one by one.

Reading one character at a time from disk is slow, so buffering is used.

From notes:

Special buffering techniques are developed to reduce time required to process input characters. Buffer is divided into two halves. Two pointers are used: `lexemeBegin` and `forward`.

■ Two-Buffer Scheme

Buffer divided into two parts:

```
| Buffer 1 | Buffer 2 |
```

Each buffer contains N characters.

Two pointers:

lexemeBegin → start of lexeme

forward → scans characters

■ Diagram

Buffer:

```
-----  
| part1 | part2 |  
-----
```

`lexemeB` → start

`forward` → moves

From handwritten diagram pages showing buffer pairs

■ Working Algorithm

```
if forward reaches end of first half
```

```
load second half
```

```
if forward reaches end of second half
```

```
load first half
```

```
forward = forward + 1
```

■ Sentinel Technique

To avoid checking end of buffer each time, EOF marker is placed at end.

```
| data | data | EOF |
```

Advantages:

- faster scanning
- less comparisons

■ Advantages of Input Buffering

- ✓ Faster lexical analysis
- ✓ Less I/O operations
- ✓ Efficient token recognition

2 Transition Diagrams (Intermediate Level)

Definition

Transition diagram is a graphical representation of DFA used to recognize tokens.

Used in lexical analyzer.

Each diagram contains:

- states
- transitions
- input symbols
- accepting state

Identifier recognition

Pattern:

```
id → letter (letter | digit)*
```

Diagram:

```
letter
(0) -----> (1)
|
| letter/digit
|
-----
```

State 1 = final state

Accepts:

```
abc
a1
var2
```

Rejects:

```
1abc
@name
```

Concept from token / pattern / lexeme section

Number recognition

```
digit+
```

Diagram:

```
digit
(0) ----> (1)
|
| digit
|
-----
```

■ Keyword vs Identifier

Transition diagram must check:

if → keyword

id → identifier

Example from token table in notes

■ Use in lexical analyzer

Source → Lexical analyzer → Tokens

Diagram from notes:

Source program → Lexical analyzer → Parser

|

Symbol table

3 Intermediate Code Generation (Intermediate Level)

Definition

After semantic analysis, compiler generates machine-independent representation.

From PDF:

Compiler generates intermediate representation which is easy to produce and easy to translate.

Properties of Intermediate Code

1. Easy to generate
2. Easy to translate
3. Machine independent

Example from notes

Source:

```
position = initial + rate * 60
```

Step 1 — Syntax tree

```
=  
/  
position +  
/  
initial *  
/  
rate 60
```

From diagram page

Step 2 — Intermediate code (Three address code)

```
t1 = inttofloat(60)  
t2 = id3 * t1  
t3 = id2 + t2  
id1 = t3
```

Three address code format

```
x = y op z  
x = op y  
x = y
```

Example:

```
t1 = b * c  
t2 = a + t1
```

■ Role in compiler

Source

↓

Syntax analysis

↓

Semantic analysis

↓

Intermediate code

↓

Optimization

↓

Code generation

From phase diagram

4■■■ Code Optimization (Intermediate Level)

■ Definition

Code optimization improves intermediate code to generate better target code.

From PDF:

Optimization makes code faster, shorter, and efficient.

■ Example from PDF

Before optimization:

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

After optimization:

```
t1 = id3 * 60.0
id1 = id2 + t1
```

- ✓ fewer instructions
- ✓ faster execution

■ Types of Optimization

1. Constant folding

```
x = 2 + 3
→ x = 5
```

2. Dead code elimination

```
x = 5
x = 6
```

Remove first.

3. Common subexpression

```
a = b*c
d = b*c
```

→

```
t = b*c
a = t
d = t
```

4. Loop optimization

```
for i
x = y + z
```

Move constant outside loop.

5. Strength reduction

```
x = y * 2
```

```
→ x = y + y
```

■ Optimization phase diagram

```
Intermediate code
```

```
↓
```

```
Code optimizer
```

```
↓
```

```
Optimized code
```

```
↓
```

```
Code generator
```

From module diagram

■ Next Steps

■ If you want next level

You can explore:

- ✓ ADVANCED LEVEL topics
- ✓ More theory and algorithms
- ✓ DFA construction in detail
- ✓ Quadruples and triples representation
- ✓ Advanced optimization techniques
- ✓ Compiler bootstrapping
- ✓ Compiler phases at deep level

For more detailed explanations, check the advanced level materials.

This is an intermediate-level study guide for Module 1 - Compiler Design. It provides deeper understanding than basic level while maintaining clarity for students transitioning to advanced concepts.